Training a Playlist Curator Based on User Taste

Machine Learning Mini-Project Write-up | PES University, Bengaluru

Purushothama K C, SRN PES1UG25AM810 | Raghav G Uddihal, SRN PES1UG24AM215

1. Problem Statement

Building playlists by hand does not scale: a listener with thousands of songs has to decide where each one belongs, and song titles or genre labels capture only part of what makes a playlist feel coherent. Commercial systems lean on collaborative filtering across many users, which says little about how one person groups songs. This project treats playlist curation as a **supervised multi-class classification** problem: given a track's vector of *n* audio features, predict which of the *C* playlists it belongs to. As a second task, it recommends the songs **most acoustically similar** to a chosen song. The work is inspired by Awadelkarim and Coelho (Stanford, 2018), *Training a Playlist Curator Based on User Taste*, who classified songs into playlists with audio features and genre tags.

2. Dataset Details

The dataset (`spotify_playlists.csv`) has **5,721 tracks** and 17 columns: identifiers and metadata (*track_id*, *track_name*, *artist_name*, *genre*, *duration_ms*), the target *playlist*, and **11 audio features**. Danceability, energy, speechiness, acousticness, instrumentalness, liveness and valence lie in 0–1; loudness is in dB (−23.8 to −1.0), tempo in BPM (55–178), key is 0–11 and mode is 0/1. The target has **12 classes**: Others (821 tracks), Chill (600), Workout (550), Party (520), Romantic (480), Sad (460), Acoustic (440), Focus (410), Travel (390), Jazz (370), Rock (350) and Hip Hop (330). The data has no missing values and no duplicate rows, but the classes are moderately imbalanced (largest class is about 2.5× the smallest), and "Others" is a catch-all category. By comparison, the Stanford paper used a 1,044-song, 13-class set with 2,105 features, so its accuracy is not directly comparable to ours.

3. Approach

Preprocessing. Rows with missing features or labels are dropped (none here), playlist names are label-encoded, and the data is split **80/20 with stratification** (4,576 training and 1,145 test songs, seed 42). A **StandardScaler** is fitted on the training set only and then applied to the test set, which avoids data leakage. Scaling matters because loudness (dB) and tempo (BPM) have much larger ranges than the 0–1 features and would otherwise dominate distance-based models.

Models. Five classifiers from scikit-learn were trained on the same scaled features and compared: Logistic Regression (max 1,000 iterations); K-Nearest Neighbors (k = 5); Random Forest (100 trees); SVM with RBF kernel (C = 1.0); and a Multi-Layer Perceptron (hidden layers 64 and 32, at most 300 iterations). Each model is scored on the held-out test set with accuracy and weighted precision, recall and F1, plus 5-fold cross-validation on the training set. The model with the highest test accuracy is saved as the best model.

Recommendation. Songs are compared in the same scaled feature space using **cosine similarity**, Similarity(A, B) = (A · B) / (|A| × |B|). The system ranks all other songs by this score and returns the top N.

4. Implementation Overview

The project is a full-stack application. The **backend** (Python, FastAPI, Uvicorn) is split into modules for data loading, preprocessing, model training, evaluation, prediction, recommendation and PCA. It exposes REST routes for dataset upload, summary and preview; EDA (class distribution, correlation, PCA, scatter, feature statistics); model training, comparison and per-model metrics; prediction; recommendation; and model insights. Trained models, the scaler and the label encoder are saved with Joblib so predictions reuse exactly the same preprocessing. The **frontend** (React, Vite, Tailwind CSS, Recharts) has pages for dataset analysis, EDA, model training, model comparison, playlist prediction, song recommendation and model insights. The prediction page takes feature sliders (or an existing song) and shows the predicted playlist with its confidence and the top-5 class probabilities. The analysis was first done in four Jupyter notebooks (exploration, preprocessing, training, evaluation).

5. Results

Test-set results on the 1,145 held-out songs (precision, recall and F1 are weighted averages):

Model	Test accuracy	Precision	Recall	F1-score	5-fold CV (train)
SVM (RBF)	92.14%	0.923	0.921	0.922	89.62% ± 0.68
Neural Network (MLP)	91.88%	0.919	0.919	0.919	88.68% ± 0.88
Random Forest	91.35%	0.915	0.914	0.913	88.44% ± 0.50
Logistic Regression	89.87%	0.900	0.899	0.899	87.50% ± 0.76
K-Nearest Neighbors	85.07%	0.861	0.851	0.847	82.93% ± 0.72

Table 1. Test-set performance of the five classifiers (values from the saved evaluation output trained_models/metadata.json; re-running the training code with the same seed reproduced them exactly).

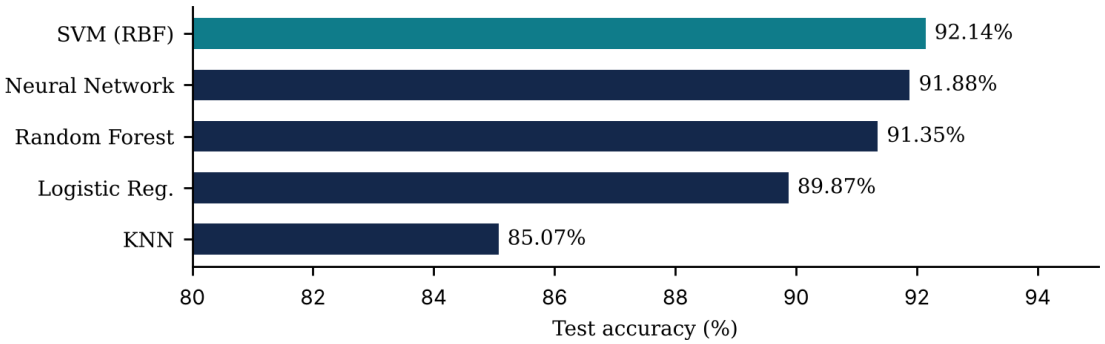


Figure 1. Test accuracy of each model on the held-out 20% split.

- **SVM (RBF) is the best model** at 92.14% test accuracy, closely followed by the MLP (91.88%) and Random Forest (91.35%). The gaps among these three are small (under one percentage point), so on one split they should be regarded as roughly equal. KNN is clearly the weakest.
- **Scaling is essential.** Without StandardScaler, SVM accuracy falls from 92.1% to 39.0% and KNN from 85.1% to 41.3% on the same split.
- **Per-playlist behaviour (SVM).** Hip Hop (F1 0.99), Focus (0.99) and Rock (0.98) are easiest. Acoustic (precision 0.80), Chill (F1 0.86) and Romantic (F1 0.89) are hardest, as their audio profiles overlap with neighbouring moods. In the Random Forest, the most important features are instrumentalness (0.15), acousticness (0.15) and energy (0.13).

6. Conclusions

Audio features alone are enough to sort songs into 12 playlists with about 92% accuracy using an RBF-kernel SVM, and standardizing the features is the single most important preprocessing step. The project delivers a working end-to-end system: a reproducible training pipeline, a model comparison, a playlist predictor that reports confidence, a cosine-similarity recommender and an interactive dashboard. Limitations are that results come from a single stratified split (cross-validation scores are about 2–3 points lower than test accuracy), that the catch-all “Others” playlist is inherently ambiguous, and that the recommender measures acoustic closeness, not a listener's personal taste. Future work includes using listening history and user feedback, combining content-based and collaborative filtering, and adding mood-based playlist generation.

Reference: *A. Awadelkarim and K. Coelho, “Training a Playlist Curator Based on User Taste,” Stanford University, 2018.*